VC Audit Tool

Fair Value Estimation for Private Portfolio Companies

Vishaal Kumar · March 2026

Modus Audit — Software Engineer

Problem & Approach

Problem

Auditors need to estimate fair value of private, illiquid portfolio companies in VC portfolios

Challenge

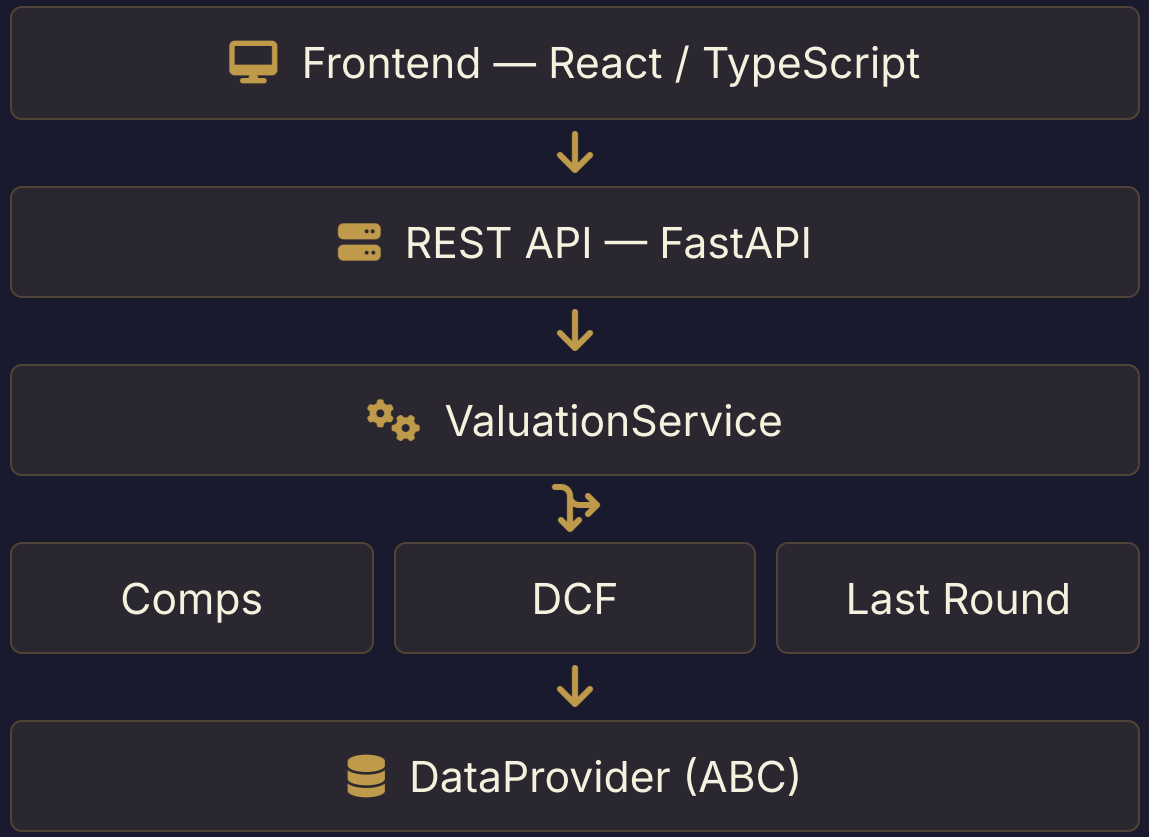
No market price — need structured, defensible methodology with a complete audit trail

My Approach

- Full-stack service with **3 valuation methodologies**
- Every calculation step recorded for auditability
- Sensitivity analysis & multi-method triangulation

Python / FastAPI | React / TypeScript | Pydantic | Tailwind CSS

Architecture Overview



Key Principles

- **Strategy pattern** for pluggable valuation engines
- **Dependency injection** — DataProvider abstraction decouples data from logic
- **Single responsibility** — each layer has one job
- **Pydantic models** as single source of truth for validation, serialization, and OpenAPI docs

Valuation Engines

↓

Comparable Company Analysis

How it works

- Find public comps by sector
- Calculate EV/Revenue multiples for each
- Apply **median multiple** to target company revenue

Implementation

- `statistics.median()` for central estimate
- `quantiles()` for Q1/Q3 range
- Minimum 2 comparables required (raises `ValueError`)

Mock Data

15 real public companies across 4 sectors

Datadog, Snowflake, CrowdStrike, Palantir, UiPath...

Output

Point estimate + interquartile range

Step-by-step derivation with citations

Discounted Cash Flow (DCF)

How it works

- Project 5-year free cash flows
- Discount at WACC to present value
- Add terminal value via **Gordon Growth Model**

Key Formulas

- Year N FCF = Revenue × (1+g)N × margin
- Terminal = FCFfinal × (1+gt) / (WACC − gt)

Validation

Cross-field: `discount_rate` must exceed `terminal_growth_rate`

Range: ±20% sensitivity bounds

Why DCF Matters for Audit

Most assumption-heavy methodology — most important to make every input **transparent and traceable**

Last Round (Market-Adjusted)

How it works

- Start from last funding round valuation
- Adjust by **public index performance** since that date
- Simplest method — provides useful anchor point

Implementation

- Uses Nasdaq index data (mock: Jan 2023 – Mar 2026)
- Index return = (current – round_date) / round_date
- Range: ±15% for illiquidity / company-specific risk

Date Validation

Requires index data within **180 days** of round date

Raises error if no suitable data point found

Role in Triangulation

Provides market-based anchor to cross-check model-driven estimates from Comps and DCF

Auditability

Audit Trail & Traceability

Core of the assignment — auditors need defensible, traceable valuations

ValuationResult Structure

1. `steps[]` — numbered derivation steps with description + detail
2. `assumptions{}` — every input and intermediate value recorded
3. `citations[]` — data sources for every piece of external data

PDF Export

Summary table + per-method breakdowns

Full audit narrative with all assumptions

Generated via `ReportLab`

Design Principle

Every calculation step is recorded — not just the final number. An auditor can trace any estimate back to its inputs.

Sensitivity & Triangulation

Sensitivity Analysis

- **DCF:** 2D grid (discount rate × growth rate) with color-coded heatmap
- **Comps / Last Round:** 1D parameter sweep
- Shows how the estimate changes with different assumptions

Why It Matters

Auditors want to know: “How sensitive is this number to my assumptions?”

Triangulation (Multi-Method)

- Run all 3 methods simultaneously
- Comparison table: method, estimate, range
- Visual range bars with **convergence zone** overlay

Convergence

If ranges from multiple methods overlap → higher confidence in the estimate

Divergence flags areas needing closer review

Quality & UX

Testing Strategy

~50+ tests across all layers

Models

- Pydantic validation
- Cross-field constraints
- Serialization round-trips

Engines

- Correct calculations
- Edge cases (< 2 comps)
- Monotonicity properties

Service

- Multi-method orchestration
- Report assembly
- Error propagation

API

- Full request/response cycles
- Error codes & validation
- PDF export (magic bytes)

Sensitivity

- Grid generation
- Parameter monotonicity
- Boundary conditions

Pattern

- Custom test providers
- e.g. `_FewCompsProvider`
- Isolated edge case testing

Frontend Walkthrough

Two-Panel Layout

- Sticky input form (left) + scrollable results (right)
- Smart form: conditional fields based on selected methodologies
- Client-side validation mirrors backend constraints

Rich Result Display

- Per-method cards: estimate, range, assumptions, audit trail
- Sensitivity toggle per method
- Triangulation view when multiple methods selected

Validation Examples

Positive revenue required

Discount rate > terminal growth rate

At least one methodology selected

React 18 | TypeScript | Tailwind CSS | Vite

Key Design Decisions

Strategy Pattern

Pluggable valuation engines — easy to add new methodologies without touching existing code

Pydantic as Source of Truth

One model definition drives validation, serialization, and OpenAPI documentation

DataProvider ABC

Mock data today, real APIs tomorrow — swap without touching business logic

Audit-First Design

Every calculation step recorded, not just final numbers — built for traceability from the start

Multi-Method Triangulation

Convergence analysis across methods increases confidence — divergence flags areas for closer review

Thank You

Questions about design, implementation, or methodology?

Vishaal Kumar · vishaal@hey.com